

deepresearch agent

姓名：林丞毅

学号：231300025

目录

1 任务概述	3
1.1 任务描述	3
1.2 核心挑战	3
2 Baseline: ReAct 状态机 Agent	3
2.1 轨迹错误分析	4
2.1.1 类型一：查询词无区分度（5 题）	4
2.1.2 类型二：区分性约束未转化为查询（4 题）	4
2.1.3 类型三：实体过早锁定（3 题）	5
2.1.4 关键发现	5
3 实验设置与评估框架	6
3.1 数据集	6
3.2 基座模型	6
3.3 检索设置	6
3.4 超参数	6
3.5 实验结果	6
4 OpenTrack 额外提交	7
4.1 设计动机	7
4.2 代码框架设计	7
4.3 四阶段流水线流程图	8
4.4 新工具设计	8
4.4.1 ConstraintTracker（约束追踪器）	9
4.4.2 ReRanker（检索结果重排器）	9
4.4.3 多路查询生成器（Query Planner）	10
4.5 提示词设计	10
4.6 上下文管理	11

4.7	终止条件	11
4.8	与 ReAct 的核心区别与优势	11
4.9	轨迹错误分析	13
4.9.1	类型一：查询词未覆盖关键区分性约束（5 题）	13
4.9.2	类型二：实体过早锁定（3 题）	14
4.9.3	类型三：约束分散无单文档全量覆盖（2 题）	14
4.9.4	类型四：信息粒度过细（1 题）	14
4.9.5	关键发现	14
4.10	消融对比	15
4.11	监督微调尝试（未采用）	16
4.11.1	方案设计	16
4.11.2	设计动机	17
4.11.3	实验结果与放弃原因	17
5	附录	19
5.1	A. ReAct 系统提示词（react_loop.py）	19
5.2	B. OpenTrack 各 Agent 提示词（multiagent.py）	19
5.3	C. 参数设置	20

1 任务概述

1.1 任务描述

BrowseComp-Plus 要求智能体针对一个复杂问题，通过多轮检索文档语料库找到精确答案。和普通的问答任务不同，这里的问题通常包含 3-5 个独立约束条件，必须跨文档推理才能同时满足。而且 BM25 检索会召回大量只匹配了单个常见词的干扰文档 (hard-negative)，单次搜索很难覆盖所有约束，智能体需要在多轮交互中逐步收集和筛选证据。

1.2 核心挑战

实际跑下来，这个任务有几个比较棘手的地方。首先是 BM25 的召回率瓶颈——它只看词频，语义相近但用词不同的文档根本搜不到，智能体必须自己想办法生成多样化的查询。其次是噪声问题，hard-negative 的设计让 top-k 结果里干扰文档占比很高，一旦从错误文档里提取了事实，后续推理就会被带偏。另外多轮搜索中模型还要维护已确认事实、已搜过查询、已排除线索这些状态，但 LLM 上下文窗口有限，早期信息容易被挤掉。并且，LLM 有时候无法分解出很好的 query 进行搜索。最后是过早终止的问题，模型经常在证据不足时就急着给答案，尤其是连续几轮搜不到东西的时候。

2 Baseline: ReAct 状态机 Agent

ReAct (Reasoning + Acting) 的核心理念是让模型在 Thought $\rightarrow$ Action $\rightarrow$ Observation 的循环中逐步推进推理。基于此实现了带状态机的 ReAct Agent (`react_loop.py`)，用四状态有限状态机 (THINK $\rightarrow$ OBSERVE $\rightarrow$ ANSWER $\rightarrow$ DONE) 控制行为流转，通过 EvidenceStore 启发式提取关键事实实现上下文精炼，通过运行时指标检测和动态提示注入进行纠偏（如检测只搜不读、连续空搜、搜索次数不足等）。工具为 `search(query)` (BM25 检索) 和 `get_document(docid)` (获取全文)。整体流程如图 1 所示。

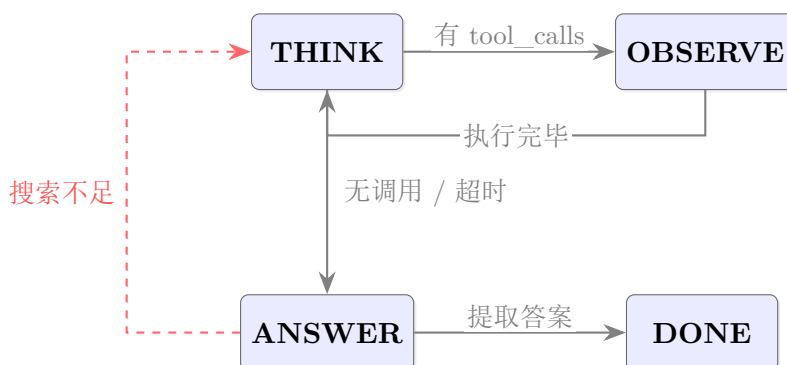


图 1: ReAct 状态机

2.1 轨迹错误分析

ReAct 在 50 题中做对 9 题、做错 41 题（准确率 18%）。从 41 道错题中选取 12 道完整阅读搜索轨迹，按根因分为三类。

表 1: ReAct 12 道错题检索失败根因分类

根因	数量	典型案例	失败机制
查询词无区分度命中无关文档	5	qid=26, 869, 180, 442, 159	BM25 字面匹配了查询中的普通词汇，返回无关文档
区分性约束未转化为查询	4	qid=216, 113, 26, 36	最具区分度的线索未被搜索，始终用泛化词检索
实体过早锁定	3	qid=216, 651, 36	搜到第一个表面匹配的实体后，后续全围绕其展开

2.1.1 类型一：查询词无区分度（5 题）

qid=26 (Binochios → Bebop) 6 次搜索全部是 “B four syllables [X]” 的变体，BM25 反复返回同一篇抑扬五音步文章（含 “syllables” 一词）。Franciscus 姓氏、Filipino southpaw boxer、\$100M 杂志等区分度线索全部未使用。Thought 中列出了所有线索，但 Action 始终停留在 “B four syllables”。

qid=869 (kindergarten → 8th grade) 搜索 “individual mid-20s dating teenager three children assault” 等 20+ 词句子，BM25 返回约会建议博客。未搜到任何真实人物文档，模型用常识推算猜了 8th grade。

qid=180 (blink-182 → The Animals) 搜索 “movie released won award” 等泛化词，“Feliformia” 这一罕见区分词从未被搜。检索结果全是通用电影列表。

qid=442 (THE DAWN → Connecticut) 搜到 Allan Cunningham（正确植物学家），但无法关联到书作者 Ida Lee。随后 “settlements early prosperity” 命中 Connecticut 历史页，被带偏。

qid=159 (Oct 15 → Oct 23) 3 次搜索后从博物馆文档推断日期，无交叉验证，差 8 天。

2.1.2 类型二：区分性约束未转化为查询（4 题）

qid=216 (Manly Hall → James Baldwin) 搜索了 “died 1990s + non-profit 1930s + pre-1940 book” 三个泛化时间约束，返回 James Baldwin。最具区分度的 “82 days after

death TV episode”从未被搜。锁定 Baldwin 后读同一文档两次，Thought 中承认 “died 1987, not 1990s” 但不改变方向。

qid=113 (Paropsis dilatata → Popillia japonica) 搜索 “invasive beetle wrongly identified 1916” 等通用描述，BM25 返回日本甲虫。专业术语 “paropsis charybdis” 从未出现在查询中。

qid=36 (AJ Auxerre → Cruzeiro) 搜索 “1-0 yellow card substitutions referees nationalities” 等比赛特征组合，BM25 返回 1990 World Cup 赛程表。锁定 World Cup 后 9 轮全围绕它展开。可定位到具体球员的引述线索未被使用。

qid=556 (Amos Makarau → Maurice Jarre) 搜索 “southern hemisphere jazz bassist born” 直接查找人物，被著名配乐家 Jarre 的文档吸引。学术背景线索 (master thesis 1990s neighborhood oceanography acknowledgements) 完全未被搜索。

2.1.3 类型三：实体过早锁定 (3 题)

qid=651 (FormFactor → Swift Transportation) 搜到 Swift Transportation 的 class action 文档后，第 4 次搜索直接搜 “Swift Transportation Co., Inc. customer concentration”，不再验证创始人换角色、客户集中度等其余约束。

qid=815 (One Red Rose → Vanity Fair) 仅 5 次搜索全部围绕 “royal academy”，返回 Chris Hammond 插图列表。锁定 Vanity Fair 后明知作者生于 1811 (非 1860s)、父亲是银行职员 (非拍卖师)，仍输出该书名。

qid=827 (Edwin Hughes Kendrick → Samuel) 搜索方向从泛化人物特征逐步缩小，但从未搜到包含完整姓名的文档，最终猜了 “Samuel” 这一不完整信息。

2.1.4 关键发现

1. **查询生成是最薄弱环节：**12 题中 9 题涉及查询问题。Thought 能列出约束，Action 却选通用词。Feliformia、Franciscus、barrel-shaped vessel、82 days 等罕见区分词几乎从未进入查询。
2. **EvidenceStore 的正反馈锁死：**错误实体被提取后反复出现在摘要中，模型被持续提醒该实体，无法自我纠正。
3. **搜索预算不足放大错误：**平均 5.3 次搜索，方向错误时没有足够轮次纠正。

3 实验设置与评估框架

3.1 数据集

使用 BrowseComp-Plus 数据集，测试集包含 50 道多约束问题。每道题包含多个约束条件和 hard-negative 干扰项，需要跨文档推理才能找到精确答案。

3.2 基座模型

基座模型为 Qwen 系列，通过 vLLM 本地部署。推理参数：temperature=0.0, max_tokens=4096，采用 OpenAI 兼容的 function calling 接口。

3.3 检索设置

BM25 索引基于 SQLite FTS5 构建。每次 search 调用返回 top-5 结果，每条结果包含 docid、BM25 评分和文档摘要片段。

3.4 超参数

表 2: ReAct 超参数设置

参数	值	含义
max_turns	5	最大 THINK 轮数
max_history_msgs	6	上下文中保留的最近消息数
min_searches	2	最少搜索次数
max_total_calls	15	总工具调用硬上限
max_recent_chars	30000	最近消息字符预算
max_tokens	4096	模型单次生成上限

3.5 实验结果

表 3: 两种方案准确率与效率对比

方案	准确率	平均工具调用	平均检索文档数	代码
BaseReact	12% (6/50)	3.20	10.52	react_agent.py
ReAct	18% (9/50)	5.32	17.58	react_loop.py

从 BaseReact 12% → ReAct 18% ，准确率显著提升，

4 OpenTrack 额外提交

4.1 设计动机

ReAct 的准确率瓶颈源于”模型自主决策”这一根本设计——模型同时承担搜索规划、文档筛选、事实提取、答案推理四个角色。一旦初始查询方向错误或模型被 hard-negative 文档的表面匹配吸引，后续搜索就在错误路径上打转，EvidenceStore 的摘要又进一步固化错误方向，形成”错误锁定”的恶性循环。ReAct 的纠偏提示只能事后补救，无法从结构上解决”单点决策风险”。并且根据对输出的观察，发现 ReAct 中，模型调用工具的过程非常不稳定。

OpenTrack 的设计理念是将这四个角色解耦为独立的专业化 Agent，由 Orchestrator 代码硬编码串联执行，将搜索策略从“模型自主决定”转变为“代码驱动的多路召回 + 多 Agent 协作验证”。核心改进包括：

1. **多 Agent 分工**：ScreenAgent（筛选）→ ExecutorAgent（提取）→ AssessorAgent（评估 + 重规划）→ SynthesizerAgent（回答），每个 Agent 只需聚焦单一任务，提示词高度专业化。
2. **多路查询召回**：每轮生成多条互补 BM25 查询，扩大关键词覆盖空间，避免 ReAct 单条查询方向错误的致命影响。
3. **ReRanker 二次排序**：对 BM25 原始结果进行多特征重排（问题关键词覆盖、约束匹配度、clue 片段命中率、已读惩罚），将 hard-negative 推后。
4. **约束追踪**：从问题提取原子约束，逐轮追踪 verified/partial/missing 状态，作为 AssessorAgent 判断”何时停止”的量化指标。不过最终结果表明，提取正确约束也是影响准确率的一大问题。

4.2 代码框架设计

框架由 4 个 Agent + 2 个工具类 + 查询规划模块 + Orchestrator 构成：

- **四个 Agent**：ScreenAgent 筛选相关文档、ExecutorAgent 提取事实与排除线索、AssessorAgent 评估约束覆盖并生成查询、SynthesizerAgent 综合事实合成答案。各 Agent 仅聚焦单一任务，内部维护跨轮次状态。
- **ConstraintTracker + ReRanker**：前者提取原子约束并追踪 verified/partial/missing 状态；后者对 BM25 结果做多特征线性重排（6 维 + 4 项 bonus），将 hard-negative 推后。
- **查询规划模块**：_plan_queries（锚点 + 内容词 + 罕见词）、_browsecomp_queries（关系词 + 分段）、_constraint_queries（约束缺口补缺）三路协同。

- **Orchestrator:** 代码硬编码串联 Phase 0（约束提取）→ 循环（多路查询 → 搜索 → 筛选 → 提取 → 更新 → 评估 → 停止判断）→ SynthesizerAgent。

4.3 四阶段流水线流程图

OpenTrack 的完整执行流程如图 2 所示。与 ReAct 的”模型自主决策环”不同，该模式采用代码硬编码的纵向流水线结构——Orchestrator 在每个阶段调用对应的 Agent，Agent 之间不互相调用，所有决策都通过代码逻辑串联。

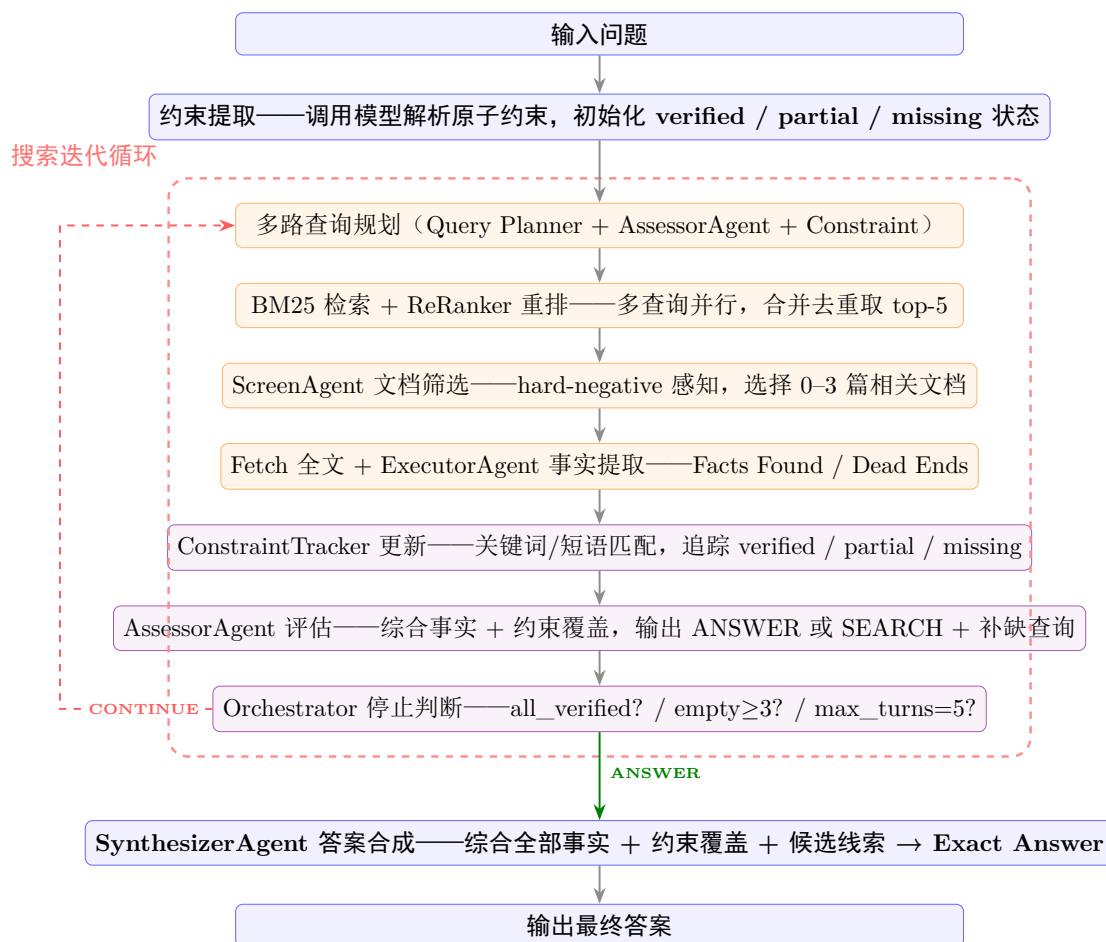


图 2: OpenTrack 多智能体框架流程图

4.4 新工具设计

相比 ReAct 仅提供 `search` 和 `get_document` 两个基础工具，OpenTrack 在检索链路上设计了三个专用工具类，将原本由模型自主决策的能力转化为代码硬编码的可靠模块。

4.4.1 ConstraintTracker（约束追踪器）

约束追踪器是 OpenTrack 区别于 ReAct 的核心数据结构。Phase 0 通过调用模型从问题中提取原子约束列表（JSON 格式，如 `["id":"C1","text":"born in France", "id":"C2","text":"won Nobel Prize in 1920s"]`），每个约束维护三种状态之一：

- **verified**: 已有事实直接匹配该约束——完整短语命中事实文本，或关键词重叠率。
- **partial**: 部分事实提及约束中的关键词，但信息不完整。
- **missing**: 尚未发现任何相关事实。

更新规则（`update()` 方法）：对每个约束，提取关键词（经停用词过滤 + 词形归一），遍历所有已累积事实计算匹配率。若约束文本作为完整短语出现在事实中，或关键词匹配数 ≥ 2 且匹配率 $\geq 80\%$ ，则标记为 `verified`；只要有任何关键词命中，则至少标记为 `partial`。

覆盖率得分（`coverage_score()`）： $\text{coverage} = (\sum_i v_i)/N$ ，其中 `verified` 贡献 1.0、`partial` 贡献 0.5、`missing` 贡献 0.0。该得分是 `AssessorAgent` 判断“是否可以停止”的核心量化指标。

此外，`ConstraintTracker` 还提供 `missing_summary()`（生成缺失约束的文本摘要，传入 `AssessorAgent` 上下文）、`open_constraints()`（返回未验证约束的文本列表，供 `_constraint_queries` 生成补缺查询）、`best_candidate()`（从事实中找出约束覆盖最多的命名实体名，作为 `SynthesizerAgent` 的候选线索）等辅助方法。

上述指标均为启发式，具体数值经过多轮测试得到，由于测试时间有限，部分可能达到最佳效果。

4.4.2 ReRanker（检索结果重排器）

`ReRanker` 对 `BM25` 返回的原始候选进行二次排序，解决 `BM25` 仅依赖词频、无法区分 `hard-negative` 的问题。评分公式为多特征线性组合（`rerank()`）：

$$\begin{aligned} \text{score} = & 0.55 \cdot \text{norm_bm25} + 0.24 \cdot q_overlap + 0.08 \cdot \text{question_overlap} \\ & + 0.07 \cdot \text{constraint_overlap} + 0.10 \cdot \text{clue_coverage} + \text{bonus} \end{aligned}$$

各特征含义：

- **norm_bm25**: 原始 `BM25` 得分 Min-Max 归一化到 $[0, 1]$ ，保留基础词频信号。
- **q_overlap**: 当前查询词与文档文本的归一化词集合重叠率，确保搜索方向不被重排完全改变。
- **question_overlap**: 问题全文内容词与文档的重叠率，扩大语义覆盖。

- **constraint_overlap**: 未验证约束文本与文档的重叠率，优先推送可能补全约束的文档。
- **clue_coverage**: 从问题拆分的 clue 片段（引号内容、括号内容、关系从句）被文档命中的比例，鼓励同时覆盖多个独立线索的文档。
- **bonus**（四项加分）：锚点词 +0.06、精确短语 +0.08、多线索（命中 2+ clue 片段）+0.06、freshness（未读 +0.04 / 已读 -0.08）。

4.4.3 多路查询生成器（Query Planner）

查询生成模块负责将长问题转化为多条互补 BM25 短查询，是“多路召回”策略的工程核心。首轮采用“锚点 + 内容词 + 罕见词”三路组合：

1. 从问题中提取**锚点词**（引号内容、专有名词、年份，如“barrel-shaped vessel”、“Franciscus”、“1920s”），采用正则 `_extract_anchors()` 函数。
2. 提取**内容词**（去 `_STOPWORDS` 停用词词典过滤后的实词，最多 18 个），按出现位置分为前段、中段两路。
3. 提取**罕见词**（按词长降序排列，优先选长词——长词通常在语料库中更独特）。
4. 组合生成 4 条候选查询：锚点 + 内容词前段、纯内容词前段、内容词中段、锚点 + 罕见词。

此外，`_browsecomp_queries()` 针对 BrowseComp 多 clue hard-negative 题型额外生成互补查询，利用问题分段（`_question_segments()` 按关系从句拆分）、约束文本和关系词（`_RELATION_HINTS`，包含 author/writer/spouse/born/died/novel/award 等 83 个关系指示词）生成多角度查询。

后续轮次则是**混合模式**：AssessorAgent（模型）基于当前事实与约束缺口生成 2-3 条查询，经 `_clean_query()` 去停用词、截断到 2-6 词后，再叠加 `_constraint_queries()`（启发式）从 ConstraintTracker 取未验证约束提取关键词拼成补缺查询，两者合并去重后每轮最多 4-5 条。查询生成整体是**首轮启发式、后续模型生成 + 启发式清洗 + 启发式补缺**的混合管线。

4.5 提示词设计

四个 Agent 各有独立角色提示词：**ScreenAgent**（筛选）采用 hard-negative 感知策略，优先多约束覆盖的文档；**ExecutorAgent**（提取）分类输出 Facts Found 与 Dead Ends；**AssessorAgent**（审计）遵循先检查再搜索 → 链式推进 → BM25 规范 → 三元组输出五条规则，提供 `rethink()` 换方向方法；**SynthesizerAgent**（综合）仅基于已确认事实回答。

4.6 上下文管理

与 ReAct 的 EvidenceStore 精炼摘要不同，OpenTrack 采用分层记忆 + 按需注入策略：ExecutorAgent 维护跨轮 facts/dead_ends（传给 Agent 时取最近 15 条）；AssessorAgent 维护查询历史 qh 避免重复搜索；ConstraintTracker 维护约束状态并注入 missing_summary() 到 AssessorAgent/SynthesizerAgent；ScreenAgent 维护已读 docid 去重；_fetch() 限制每个 docid 最多读 2 次；每个 Agent 调用上下文截断在 20000 字符内。

4.7 终止条件

五层终止机制：轮次上限（max_turns）硬性兜底；全部约束 verified 理想终止；AssessorAgent 输出 ANSWER 且约束满足；连续空搜 3 轮（先 rethink() 换方向，仍空搜则放弃）；无更多可搜索查询自动终止。

4.8 与 ReAct 的核心区别与优势

本质差异在于控制范式的转变——从”模型自主决策”到”代码硬编码控制”，通过多智能体分工降低单点失败风险。表 4 从七个维度进行了对比。

表 4: ReAct 与 OpenTrack 核心设计对比

维度	ReAct	OpenTrack
决策控制	模型自主决定搜索/阅读/停止	Orchestrator 代码硬编码串联, Agent 只做各自任务
Agent 数量	1 个 (模型同时承担所有角色)	4 个 (Screen / Executor / Assessor / Synthesizer)
每轮查询数	1 条 (模型生成)	2-4 条 (多路互补: Query Planner + Assessor + Constraint)
检索重排	无, 直接使用 BM25 排序	ReRanker 多特征重排 (6 维特征 + 4 项 bonus)
状态追踪	EvidenceStore (启发式事实提取, 无约束概念)	ConstraintTracker (原子约束 + verified/partial/missing 状态 + coverage score)
上下文管理	精炼摘要替换 (EvidenceStore 摘要替换原始数据)	分层记忆按需注入 (每个 Agent 仅接收精简上下文, 20000 字符预算)
纠偏方式	动态提示注入 (事后纠偏, 5 种触发条件)	结构化 Agent 分工 (事前预防, 每个 Agent 只做一件事) + 约束缺口驱动查询规划
停止判定	模型自主 + 最低搜索次数硬检查	约束全部验证 + AssessorAgent 评估 + 空搜次数 + 轮次上限
优势	简单、灵活、工具调用少 (平均 5.3 次)	系统性强、覆盖广、不易被单一 hard-negative 误导
劣势	容易过早终止、被 hard-negative 误导后难以纠偏、单次失败即全局失败	工具调用多 (平均 36.7 次)、成本高、多 Agent 叠加可能引入新的协调偏差

OpenTrack 的核心优势可归纳为三点:

1. **多路召回降低单点风险:** ReAct 每轮只有 1 条查询, 若关键词选择错误则整轮白费。OpenTrack 每轮生成 2-4 条互补查询并行搜索, 即使个别查询方向偏差, 其他查询仍可能命中目标。这是准确率从 18% 提升到 30% 的最主要因素。
2. **专业化分工提升子任务质量:** 将”判断读哪篇””从文档提取什么””是否继续搜””最终答案写什么”四个任务分配给四个独立 Agent, 每个 Agent 的提示词可以高度针对其子任务优化, 避免 ReAct 单模型在多目标间冲突 (如”既要继续搜又怕超时”)。

3. 约束追踪提供可量化的停止信号：ReAct 的停止完全依赖模型判断（+ 最低搜索次数硬检查），模型经常在证据不足时过早停止。OpenTrack 的 ConstraintTracker 提供 coverage_score（0–1 分）和 missing_summary，AssessorAgent 可以基于量化指标而非直觉决定何时停止，显著减少了 ReAct 的“理解偏差”（73.2% → 37.1%）。

4.9 轨迹错误分析

本次 OpenTrack (submission_34) 在 50 题中做对 17 题、做错 33 题（准确率 34%）。从 33 道错题中选取 11 道完整阅读轨迹，按根因分为四类。

表 5: 11 道错题检索失败根因分类

根因	数量	典型案例	失败机制
查询词未覆盖关键区分性约束	5	qid=432, 442, 815, 869, 26	生成的查询遗漏了最独特的线索，命中泛化文档
实体过早锁定致方向单一	3	qid=36, 216, 427	首轮选中错误实体后，链式推进导致后续全围绕该实体搜索
约束分散无单文档全量覆盖	2	qid=314, 395	多条约束分散在语料库各处，单文档筛选策略失效
信息粒度过细 BM25 无法定位	1	qid=471	答案埋在致谢段落 1–2 句话中，全文检索无法区分

4.9.1 类型一：查询词未覆盖关键区分性约束（5 题）

qid=432 (Aaron Strick → 无法确定) 28 次 tool_calls（全 50 题最多）。10+ 条约束各搜各的——linguistics degree 1869、kenya fieldwork、city plant renamed saint——每轮返回碎片信息，但缺乏“人名”连接键，AssessorAgent 无法跨轮关联“语言学学位某人”和“定居改名城市某人”是否为同一人。

qid=26 (Binochios → Bossa Nova) 成功搜到 Franzini 姓氏（源于 Franciscus）和 Joe Noynay (Filipino southpaw 拳手)，但包含“Binochios”的文档始终未被命中。每条线索单独都搜到了，但查询未能覆盖该俱乐部独有的特征组合。

qid=815 (One Red Rose → Vanity Fair) 仅 5 次搜索全部围绕“royal academy”——一个高频通用词。“author born 1860s”、“parent was auctioneer”、“23 books 1888–1901”等区分度约束全部未出现在查询中。

qid=869 (kindergarten → 6th grade) 查询如“individual mid-20s dating teenager three children”完全是人生故事描述，BM25 返回约会博客和 CDC 研究。这类短语不会直接出现在描述真实人物的文档中。

qid=442 (THE DAWN → 无法确定) 成功找到正确书籍和作者 Ida Lee, 但 “chapter 1 title” 的字面搜索无法命中。目录页或详细摘要需要不同的检索路径。

4.9.2 类型二：实体过早锁定（3 题）

qid=36 (AJ Auxerre → Slavia Prague) 首轮选中 Josef Bican 后, 连续 9 轮查询全部含 “josef bican”, 即使 ExecutorAgent 每轮报告 Facts Found: None。AssessorAgent 的链式推进在实体错误时变成自我强化锁定。

qid=216 (Manly Hall → John Steinbeck) 锁定 Steinbeck 后不再搜索其他候选人。ExecutorAgent 标注 “died 1968, not 1990s” 但仍不改变方向。

qid=427 (Guerlain → Pabst) 首轮 ExecutorAgent 标注 Pabst 成立于 1844 (<1850), 但此后 8 轮全部围绕 Pabst 深入搜索。Pabst 在语料库中拥有大量文档, 高频存在使系统感觉证据充分, 忽略了最早标注的硬约束违反。

4.9.3 类型三：约束分散无单文档全量覆盖（2 题）

qid=314 (Spero → 无法确定) 三条约束 (2022 重组裁员、2023 收 3000 万现金、35 员工含 10 博士) 被分别搜索, 但无单篇文档同时命中全部三条。ScreenAgent 的单文档筛选策略在此失效。

qid=395 (Isgro → Marina C. Isgro) 成功检索到正确论文, 但 “四个连续辅音的姓氏” 被关联到论文作者而非写信人——语法修饰关系被误解析。检索正确但主体归属提取失败。

4.9.4 类型四：信息粒度过细（1 题）

qid=471 (Scott A. Ebers → 无法确定) 成功定位到正确作者 Valerie Martinez-Ebers, 但她 1990 年博士论文致谢中的丈夫姓名仅在 1-2 句话中出现。BM25 无法区分 “提到此人” 和 “包含致谢细节” 的文档。

4.9.5 关键发现

1. 查询未覆盖最独特线索是最大瓶颈 (5/11): 最具区分度的约束 (独特年份、专有名词、罕见关系词) 常被遗漏。
2. 实体锁定缺自纠正 (3/11): 链式推进在锁错实体时需增加 “连续 N 轮无发现 → 丢弃重新搜索” 的逻辑。
3. 跨文档约束合并是盲区 (2/11): 多约束分散时, ScreenAgent 的单文档策略失效。

4. **Synthesizer 行为不一致放大错误**：同样检索失败，qid=26/869/815 虚构答案，qid=432/442 保守输出无法确定。

4.10 消融对比

五种方案构成逐项消融实验——每增加一项技术，准确率随之变化：

表 6: 消融实验：从 BaseReact 到 OpenTrack 完整版的逐项增益

方案	准确率	Δ	代码	新增/移除的技术
BaseReact	12%	—	react_agent.py	基准线：纯 ReAct，模型完全自主
ReAct	18%	+6%	react_loop.py	+ 状态机 + EvidenceStore + 纠偏提示
架构消融	22%	+4%	multiagent_fc.py	+ 多 Agent 架构 (FC 调用方式)
工具消融	24%	+2%	multiagent_search_only.py	+ConstraintTracker，但无多路召回、无 ReRanker
OpenTrack	34%	+10%	multiagent.py	+ 多路召回 + ReRanker (在 消融 B— 基础上)

关键消融证据：

- **多路召回 + ReRanker 贡献 +10%**：对比工具消融（24%）与完整版（34%），两者使用相同的 4-Agent 架构和 ConstraintTracker，唯一区别是完整版增加了多路查询生成和 ReRanker 重排。这直接证明了检索增强模块对准确率的贡献。
- **多 Agent 架构贡献 +4%**：ReAct（18%）→ 架构消融（22%），两者均使用单路查询，但后者将单模型拆分为 4 个独立 Agent。
- **纠偏机制贡献 +6%**：BaseReact（12%）→ ReAct（18%），纯 ReAct 增加状态机、EvidenceStore 和纠偏提示。

4.11 监督微调尝试（未采用）

4.11.1 方案设计

错误分析显示，OpenTrack 最大的检索失败根因是查询词未覆盖关键区分性约束 (5/11)，本质上是查询生成能力不足。一个自然的想法是：能否通过监督微调 (SFT) 让

基座模型学会生成更高质量的 BM25 查询？

训练方案设计如下：

- **训练数据：**课程设计要求不能使用 BrowseComp-Plus 数据训练模型，因此使用外部数据集 **HotpotQA**（多跳问答数据集，问题需要跨多个文档推理，与 BrowseComp-Plus 的检索需求有一定相似性）。从 HotpotQA 开发集中选取 5334 条问答对，用更强模型 **DeepSeek-V4** 为每条问题生成 5 条 BM25 查询作为训练标签（知识蒸馏），构造为“问题 → BM25 查询 JSON 数组”的 SFT 格式。目的是让 Qwen3-8B 学习更强模型的查询生成策略。每条样本的输入为自然语言问题，输出为 5 条 2-6 词的英文 BM25 查询组成的 JSON 数组。例如输入 “Who was the writer of These Boots Are Made for Walkin’ and who died in 2007?”，输出 `["boots made walking writer", "author died 2007", "songwriter died 2007", "boots songwriter", "boots made walkin writer death"]`。
- **微调方式：**LoRA（Low-Rank Adaptation），参数高效微调，只训练 adapter 权重，基座模型权重冻结。配置：`rank=8`、`alpha=16`、`target_modules=["q_proj", "v_proj"]`、`dropout=0.05`。选择只微调注意力层的 query 和 value 投影，避免动 FFN 层（存知识的部分）以减少灾难性遗忘。
- **训练设置：**学习率 5×10^{-5} ，1 个 epoch，`batch_size=4`，梯度累积 4 步，`max_length=1024` tokens，`optimizer=Adafactor`，`gradient_checkpointing` 开启。
- **推理方式：**微调完成后将 LoRA adapter 合并进基座模型（`merge_and_unload`），导出为标准 HuggingFace 格式，用 vLLM 加载替换原基座模型，其余 agent 代码（ScreenAgent、ExecutorAgent、AssessorAgent、SynthesizerAgent）不变。

4.11.2 设计动机

选择只微调查询生成任务而非完整 agent 轨迹，基于以下考虑：

1. **针对最大瓶颈：**OpenTrack 的检索偏差（查询未覆盖关键约束）是最主要的失败原因（45%），如果能通过微调提升查询质量，理论上能直接提升检索命中率。
2. **保留 agent 灵活性：**ScreenAgent、ExecutorAgent、AssessorAgent 的行为逻辑由提示词控制，如果不微调这些组件，可以保持提示词可调节的优势，避免微调后行为固化。
3. **数据可获得性：**用 HotpotQA 问题 + DeepSeek-V4 蒸馏生成查询标签的方式，可以快速构造大量“问题 → 查询”训练对，而完整 agent 轨迹（包含多轮搜索、筛选、提取、评估）需要人工逐步骤标注，成本极高且外部数据集中不存在此类标注。

4.11.3 实验结果与放弃原因

微调后的模型在 50 题测试集上仅取得 **24% 准确率**（12/50），明显低于未微调 OpenTrack 的 34%。经分析，失败原因如下：

1. **灾难性遗忘**：尽管只微调了 `q_proj` 和 `v_proj`（rank=8），1 个 epoch 的训练仍导致模型遗忘了部分通用推理能力。基座模型在未微调时能正常完成 ScreenAgent 的 hard-negative 筛选、ExecutorAgent 的事实提取、AssessorAgent 的约束验证等任务，微调后这些能力显著退化。原因是 BrowseComp-Plus 的 agent 任务高度依赖模型的通用阅读理解与推理能力，而这些能力在单任务 SFT 中被弱化。
2. **训练任务与推理任务不匹配**：训练时模型只学习“输入问题 → 输出 JSON 查询数组”，但推理时模型需要完成四种不同角色的对话（ScreenAgent、ExecutorAgent、AssessorAgent、SynthesizerAgent），每种角色有独立的系统提示词和输出格式。模型在微调后倾向于在任何 prompt 下都输出 JSON 查询数组，导致非查询角色（特别是 SynthesizerAgent）的输出格式严重偏离。
3. **训练数据与目标域不匹配**：HotpotQA 虽同为多文档推理任务，但其问题特征与 BrowseComp-Plus 的多约束 hard-negative 题型差异显著。HotpotQA 的查询策略通常是“提取问题主题词直接搜索”，而 BrowseComp-Plus 需要“识别多个独立约束并分别生成互补查询”。即使使用 DeepSeek-V4 生成训练标签，DeepSeek-V4 的查询策略也是针对 HotpotQA 问题优化的，其查询模式迁移到 BrowseComp-Plus 目标域后无法直接适配。蒸馏学到的是“回答 HotpotQA 的查询策略”而非“回答 BrowseComp-Plus 的查询策略”。
4. **查询质量提升有限，其他能力损失巨大**：即使查询质量有边际提升（检索命中率略增），但 ScreenAgent 筛选错误、ExecutorAgent 提取遗漏、SynthesizerAgent 幻觉等错误模式的增加完全抵消了检索端的收益。最终 24% 的结果说明：**在 agent 系统中，微调单一子任务而破坏模型的通用能力，得不偿失。**

基于以上分析，最终方案放弃微调，直接使用 Qwen3-8B 基座模型驱动完整的多 agent 流水线。

5 附录

5.1 A. ReAct 系统提示词 (react_loop.py)

你是一个深度研究智能体。你的任务是通过搜索文档语料库来回答复杂问题。

可用工具

1. `search(query)` - BM25 关键词搜索，返回 `top-5` 结果 (含 `docid`、评分、摘要)。
 - 查询词要求：2-5 个英文实词，不要用句子或虚词。
 - BM25 是纯关键词匹配 - 只有文档中完全包含的词才能匹配。
2. `get_document(docid)` - 根据 `docid` 获取文档全文。

核心方法: Thought -> Action -> Observation 循环

Thought (推理): 分析当前状态 - 已知什么? 缺什么? 下一步?

Action (行动): 选择工具调用 - `search` 或 `get_document`

Observation (观察): 分析工具返回结果

搜索策略: 独特关键词首轮搜索 -> 线索读全文 -> 缺口换角度 -> 足够事实停止

关键: 不要只搜不读。不要重复搜索。每次查询必须用不同的关键词组合。

停止条件: 至少搜索 3 次不同角度才能放弃。每次搜索后必须读全文。

最终答案格式

Explanation: <简要推理过程>

Exact Answer: <最终答案, 专有名词或人名记得保证完整>

5.2 B. OpenTrack 各 Agent 提示词 (multiagent.py)

ScreenAgent

SYSTEM: 你是筛选智能体。审阅搜索结果, 选择与问题最相关的文档阅读。如果都不相关, 输出 `NONE`。

PROMPT: 审阅上述搜索结果, 判断哪些文档值得完整阅读。最多选择2个与题目相关的文档。

优先选择同时覆盖多个题目约束、包含具体人名/日期/作品名/地点的文档。

hard-negative 检索任务: 只匹配一个常见词或单个泛泛主题的结果通常是干扰项。

输出: Relevant DocIDs: <逗号分隔的docid, 或 `NONE`>

ExecutorAgent

SYSTEM: 你是提取智能体。从文档中提取与问题相关的具体可验证事实。只报告文档中直接陈述的内容。

PROMPT: 根据问题与完整文档文本, 分类发现:

Facts Found: 与问题相关的具体可验证事实, 每个含 `docid` 和具体细节。无相关内容写 `None`。

Dead Ends: 文档看似相关但不满足约束的排除原因。无则写 **None**。

AssessorAgent

SYSTEM: 你是审计智能体。基于已发现事实链式推进搜索，将已知实体写入查询，判断是否需要继续，并

PROMPT: 分析已确认事实和约束缺口，决定下一步。

先检查已有事实是否足以回答，能回答则输出 **ANSWER** 不要继续搜索；

否则输出 **SEARCH** 并针对缺失约束给出2-3个英文BM25查询（3-6个实词）。

链式推进：将已知实体（人名、书名、地名、机构名、年份）写入下一轮查询。

输出：**Status:** ANSWER | SEARCH。如果 **SEARCH:** Missing Constraint / Search Query / Expected Evi

SynthesizerAgent

SYSTEM: 你是综合智能体。仅基于已确认事实回答问题。

PROMPT: 所有搜索轮次已完成。根据研究过程中收集的所有已确认事实，精确回答问题。

答案尽可能完整，不过度缩写，不丢失语义。

输出：**Explanation:** <解释原因> / **Exact Answer:** <精确答案>

5.3 C. 参数设置

所有方法通过 `run_submission.py` 统一运行：`MAX_TURNS=5`, `MODEL_NAME=qwen_auto` (Qwen3-8B), `temperature=0.0`, `WORKERS=3`。

ReAct: `max_turns=5`, `max_history_msgs=6`, `min_searches=2`, `max_total_calls=15`, `max_recent_chars=30000`, `max_tokens=4096`。

OpenTrack: `max_turns=5`, `max_history_msgs=6`, `BM25 top_k=5`, `ReRanker top_k=5`, `max_docs_per_round=3`, `max_doc_reads=2`, `context_limit=20000`, `max_tokens=6000`。

SFT 训练: Qwen3-8B (student), DeepSeek-V4 (teacher), HotpotQA 5334 条, `max_length=1024`, `optimizer=Adafactor`, `lr=5e-5`, `epochs=1`, `batch_size=4`, `grad_accum=4`, LoRA `r=8` `alpha=16`, `target_modules=[q_proj, v_proj]`。